RAG Proof of Concept

Nichol Flowers
Generative AI (DS267)
July 30, 2025

Executive Summary

This proof of concept explores whether a single Retrieval Augmented Generation (RAG) system can effectively support two very different audiences: Engineering and Marketing. Despite their divergent needs (technical depth versus accessible insights), the system successfully delivered persona specific responses using a shared backend pipeline with tailored prompts. Evaluated using LLM as a Judge and RAGAS context precision, the RAG showed decent generation quality even as retrieval precision lagged. Early stage testing was done using the open source Mistral model to refine prompt design and retrieval configuration. The full batch was tested using Cohere, a more powerful hosted model, across both personas.

The preliminary proof-of-concept delivered solid generation quality even as retrieval precision appeared to trail behind. Initial tests flagged retrieval quality as the main bottleneck, but a coding error that calculated precision too early on Mistral outputs, necessitates another evaluation to accurately assess our true performance. This POC shows that with targeted refinements a single RAG pipeline can effectively serve both Engineering and Marketing.

Introduction

As organizations increasingly rely on large language models (LLMs) for internal knowledge access, the need for audience specific responses becomes critical. Engineering teams expect precise, technical detail; marketing teams need strategic, high level insights. A one size fits all system often fails both. This POC aims to evaluate whether a single RAG architecture can support both audiences by retrieving relevant documents and customizing tone and content through prompt engineering.

The system accepts natural language questions and retrieves semantically relevant context using a vector search. It then applies reranking and deduplication before generating an answer using persona specific prompts. Questions span technical model specs, benchmarks, licensing, and product summaries. The goal: accurate, grounded responses that reflect each team's expectations without building two separate pipelines.

Methodology

The RAG system was built in LangChain using a modular pipeline. Document preprocessing involved semantic chunking with token sizes ranging from 128 to 768 and overlap values from 0 to 96. The best performing runs used chunk sizes of 448 to 512 with overlaps of 50 to 64 before switching to COHERE. Vector search was performed with the 'multi qa mpnet base dot v1' embedding model. Retrieval k values were

tested from 3 to 40, with reranking applied using Cohere's 'multilingual v3.0' model. Soft deduplication and MMR filters were introduced to reduce redundancy and increase diversity.

Both personas used the same retrieval configuration but different prompt templates:

- **Engineering**: prioritized precision and technical detail, using language tailored for subject matter experts. The prompt was designed to deliver concise, factual answers.
- **Marketing:** prioritized clarity and simplicity, using language tailored for a nontechnical audience. The prompt was designed to make responses straightforward and easy to understand.

Answers were evaluated separately to ensure one persona's needs did not degrade the other's results.

Two metrics were used: LLM-as-a-Judge and RAGAS Context Precision. The judge evaluated how well the system answered each question, scoring from 1 to 5 based on factual accuracy, completeness, and clarity, essentially measuring the usefulness of the final response. RAGAS Context Precision assessed the quality of the retrieval stage by calculating the percentage of retrieved documents that were truly relevant to the question. This revealed how much of the model's input was actual signal versus noise.

Both started with Groq hosted LLaMA3, but during the final total batch run on Cohere, we switched to OpenAI's "gpt-4o-mini" for faster processing.

Key Findings

We experimented with chunk sizing (128–768), chunk overlaps (0–96), k values (3 - 40) ultimately finding that chunksize, overlap,and k-value combinations such as 448/64/20 and 512/50/8 delivered the best balance of context precision and judge score on the Mistral model.

Most judge scores fell between 3 and 4, indicating that the model reliably provided correct core facts with clear explanations and, in many cases, delivered accurate, well-structured answers that covered nearly all necessary information with only minor gaps or stylistic rough edges. This strong baseline performance exceeds expectations. Small corrections to prevent omissions and tighten the organization of the answers should elevate scores into the top tier.

We compared several embedding options and found multi-qa-mpnet-base-dot-v1 delivered the best outcomes. That said, precision across our trials was generally underwhelming with scores often landing below 0.5. After doing some evaluation against Cohere as the answer generating LLM, we uncovered a bug in our precision-calculation code. We recommend rerunning those evaluations with the fix to obtain an accurate performance picture.

The pipeline was tested using both Mistral and Cohere as the generator model. While both models produced reasonable outputs, Cohere consistently delivered stronger results. It aligned more closely with the retrieved context, reduced hallucinations, and adapted better to persona-specific prompts. Based on these improvements, Cohere was chosen as the primary model for final evaluation and reporting. These findings are based on small batch testing with Mistral followed by full batch evaluations using Cohere for both personas.

Duplicate chunks crowded the top-k context. The original ranking sometimes elevated low-signal passages over key facts. To address this, we first applied MMR-based reranking to surface diverse, high-value snippets. Early trials showed that stacking too many filters at once hurt scores. So we replaced MMR and aggressive filtering with soft deduplication.

Lessons Learned

Similar metrics don't necessarily measure the same thing. Both LLM judge and cosine similarity measure semantic relationships between generated and gold answers, but they operate at fundamentally different depths of understanding. Cosine similarity provides surface-level semantic matching by comparing vector embeddings, quickly identifying whether answers share similar topics and vocabulary. However, LLM judge evaluation performs deeper semantic reasoning. It assesses correctness, completeness, logical consistency, and contextual appropriateness; not just topical similarity. While cosine similarity might score two answers as highly similar if they discuss the same topics, an LLM judge can distinguish whether one answer is factually incorrect, incomplete, or fails to properly address the question's intent.

Expensive Metrics RAGAS context precision evaluation is inherently expensive because it requires separate LLM API calls for each retrieved document to assess relevance, but my k=30 retrieval setting made this exponentially worse. With 30 documents retrieved per question, a single question consumed 30 times more tokens for evaluation than the actual RAG generation, turning an already costly evaluation into an unsustainable compute drain. Even with a smaller k=5 setting, precision scoring would still be expensive, but my high k increased the inherent cost sixfold. The evaluation metrics requiring per-document LLM calls are costly by design, and high retrieval settings can quickly make them exceed compute budgets entirely.

The system is resilient to imperfect inputs
Even when context included irrelevant or duplicated documents, the generator still produced answers that often met persona expectations. This suggests the prompt design was strong enough to buffer against moderate retrieval noise.

Prompt engineering alone can deliver significant persona adaptation
A single pipeline with shared retrieval and embeddings can produce very different outputs just by swapping the prompt. This reduces system complexity and avoids the maintenance overhead of separate deployments.

Parameter tuning produces tradeoffs
Gains in one metric often came at the cost of another. Recognizing these tradeoffs early can save time chasing "perfect" configurations that do not exist in practice.

Challenges and Limitations

Retrieval noise Tangential and duplicate chunks dilute relevance, reducing context diversity and sometimes burying critical facts too low to influence the answer.

A "Chinchilla" query placed the only relevant document at rank 20 which was outside our context window because since the embedding model didn't assign enough importance to the uncommon word, the retriever's k was set too low, and chunking broke the keyword into low-signal fragments.

Gold answer sometimes did not answer the question Question 9 highlights a limitation where the evaluation reference omits the correct answer making it difficult to assess true performance of the system

"Who proposed the first significant statistical language model?" The gold answer did not directly address the question of "who". Instead, it references IBM running the "Shannon-style" experiments. However, the creator Claude Shannon is never mentioned in the gold answer. I accessed the actual chunk and saw the gold answer matched a specific excerpt of Wikipedia that also made no mention of Mr. Shannon. As a result, the system had no grounding to name the correct individual and therefore, returned an answer inconsistent with the gold answer.

Striking the right balance with filtering. While deduplication and MMR can improve diversity, overly aggressive filtering can remove key evidence in our experiments.

Persona prompt boundaries not always enforced. Even with explicit instructions, the model sometimes ignored the prompt instruction to only respond with "This information is not in the context." when the information was not included in the context. This might need stricter enforcement for a production setting.

Evaluation bottlenecks. Early evaluations on Groq-hosted LLaMA 3 were cost-effective but suffered long runtimes on large test sets, which slowed down early iteration. Switching to OpenAI's gpt-4o-mini sped up precision evaluation but still required careful scheduling to avoid long turnaround times.

Recommendations

This POC confirms that a single RAG system can effectively serve two distinct audiences by using prompt level customization. LLM as a Judge scores show that the system produces relevant, grounded answers even when precision is suboptimal. However, before deploying in a production setting, retrieval performance must be improved. I recommend focusing next on retrieval tuning—embedding quality, reranker calibration, and chunking strategies—while retaining the shared generation pipeline and persona prompts. With these refinements, the system could provide scalable, audience aware knowledge support across the organization.

1. **Verify Gold Answers**
   Ensure all gold answers are accurate and complete, since evaluation metrics compare generated outputs directly to these references. Flawed gold answers can distort performance measurements.

2. **Calibrate Precision**
   Rerun precision scoring to confirm the logic is functioning as expected and establish a reliable baseline before making further retrieval or reranking changes.

3. **Evaluate on Evolving Data**
   Test the system periodically on updated or changing datasets to ensure that performance holds over time, especially as source material expands or shifts.

4. **Balance Cost and Performance**
   Compare different generation models to determine which offers the best trade-off between quality and cost. Select lightweight models where appropriate to reduce evaluation overhead.

5. **Use Lightweight Metrics Where Possible**
   In addition to LLM-as-a-Judge, consider incorporating lower-cost metrics such as semantic similarity or keyword overlap for quicker tuning cycles.

6. **Roll Out in Phases**
   Implement a phased rollout to monitor system behavior and gather user feedback in a controlled environment. This approach reduces risk, enables rapid iteration, and builds stakeholder confidence ahead of full deployment.

7. **Consider Adaptive Chunking**
   Explore adaptive chunking methods that split documents at sentence or paragraph boundaries rather than fixed token lengths. This can improve retrieval precision by preserving the natural structure of information and reducing fragmentation. It also lowers long-term maintenance by minimizing the need to retune chunk size parameters as content types evolve.

Reading List

*These articles are not quoted in my paper. These are sources I used for reference to complete my assignment.

1. Encord, “What is LLM as a Judge? How to Use LLMs for Evaluation,” *Encord Blog*, 2 Jul. 2024. [Online]. Available: https://encord.com/blog/llm-as-a-judge/. Accessed: 28 Jul. 2025.

2. S. Gallagher, S. Rallapalli, and T. Brooks, “Evaluating LLMs for Text Summarization: An Introduction,” *SEI Insights*, Software Engineering Institute, Carnegie Mellon University, 7 Apr. 2025. [Online]. Available: https://doi.org/10.58012/01sa-dg66. Accessed: 3 Aug. 2025.

3. Cohere, “Reranking API,” *Cohere*, 3 Aug. 2025. [Online]. Available: https://cohere.com/llmu/reranking. Accessed: 1 Aug. 2025.

4. Hugging Face, “multi-qa-mpnet-base-dot-v1,” *Hugging Face Model Embeddings*, 2025. [Online]. Available: https://huggingface.co/model-embeddings/multi-qa-mpnet-base-dot-v1. Accessed: 3 Aug. 2025.

5. Cohere, “rerank-english-v3.0 tokenizer,” *Hugging Face*, 2025. [Online]. Available: https://huggingface.co/Cohere/rerank-english-v3.0. Accessed: 28 Jul. 2025.

6. Y. Wang *et al.*, “Impact of Gold-Standard Label Errors on Evaluating Performance of Deep Learning Models in Diabetic Retinopathy Screening: Nationwide Real-World Validation Study,” *J. Med. Internet Res.*, vol. 26, art. e52506, Aug. 2024. doi:10.2196/52506.